

■ Design Rate Limiter

System Design Interview | Distributed Infrastructure

algomaster.io | Deep Dive Notes

■ Key Requirements

Functional	Limit requests per user/IP/API key within time window; Return 429 with retry-after header when exceeded
Non-Functional	Ultra-low latency (<5ms overhead); Accurate across distributed nodes; No SPOF; Configurable rules
Scale	Handle 100K+ req/sec; Support millions of users; Rules changeable without deployment

■ Rate Limiting Algorithms

- **Token Bucket:** Bucket holds N tokens; token added at rate R/sec; each request consumes 1 token; allows bursting up to bucket capacity; memory efficient
- **Leaky Bucket:** Requests enter queue (bucket); process at fixed rate; excess requests overflow/dropped; smooths traffic but no burst tolerance
- **Fixed Window Counter:** Counter per time window (e.g., per minute); reset at window boundary; simple but edge case: 2x traffic at window boundary
- **Sliding Window Log:** Store timestamp of each request; count in last 60s window; most accurate but high memory ($O(\text{requests})$ per user)
- **Sliding Window Counter:** Hybrid: current window count + weighted prev window count; memory efficient and accurate; most common in production (e.g., Cloudflare)

■ Distributed Rate Limiter Architecture

- Middleware layer (API Gateway / Nginx module) intercepts every request before hitting application servers
- Centralized counter store in Redis: use INCR + EXPIRE commands for atomic counter ops; Redis single-threaded avoids race conditions
- Redis Lua script for atomic check-and-increment: GET count → if < limit: INCR + return allow; else: return deny
- Multi-region: each region has local Redis; accept slight over-counting (eventual consistency) vs. global coordinator (latency cost)

■ Deep Dive: Redis Implementation

- Token Bucket in Redis: store {tokens, last_refill_time} per user; Lua script atomically refills tokens based on elapsed time then checks
- Sliding Window in Redis: use ZADD with timestamp as score; ZREMRANGEBYSCORE to remove old entries; ZCARD to count; EXPIRE on key
- Redis Cluster for horizontal scaling: user_id as shard key ensures same user always hits same node

- Failure mode: if Redis unavailable → fail open (allow all) or fail closed (deny all); typically fail open with local fallback counter

■ Algorithm Comparison

Token Bucket	Sliding Window Counter
✓ Allows bursting	✓ No burst spikes
✓ Memory: $O(1)$ per user	✓ Memory: $O(1)$ per user
✓ Used by AWS, Stripe	✓ Most accurate boundary handling
✓ Smooth for most use cases	✓ Used by Cloudflare

■ Headers & Client Communication

- Response headers: X-RateLimit-Limit, X-RateLimit-Remaining, X-RateLimit-Reset (epoch seconds), Retry-After
- HTTP 429 Too Many Requests with Retry-After header for graceful client handling
- Different limits per tier: free (100/min), paid (1000/min), enterprise (unlimited) → store config in DB/config service

■ Edge Cases & Gotchas

- Race condition: two servers both read count=99, both allow request → use Redis atomic operations (INCR) or Lua scripts
- Clock skew across distributed nodes: use Redis server time (TIME command) not local server time
- Rule updates: store rules in distributed config (etcd/Consul/DB); cache locally with short TTL for performance
- Sliding window boundary: at $t=59s$, user sends 100 req; at $t=60s$ window resets, sends 100 more → 200 req in 2s; sliding window prevents this